

Name: Kaveri

Roll No: 25BCS009

Course Name: Design Analysis and Algorithm

Assignment No. 1

Aim

To understand the performance analysis of Bubble Sort, Selection Sort, and Insertion Sort algorithms.

P1. Program to Perform Sorting Using Bubble Sort

Algorithm

1. Start.
2. Read the number of elements and array elements.
3. Compare adjacent elements.
4. If the first element is greater than the second, swap them.
5. Repeat the process for all elements.
6. Continue passes until the array is sorted.
7. Display the sorted array.
8. Stop.

Python Program

```
def bubble_sort(arr):  
    n = len(arr)  
  
    for i in range(n):  
        for j in range(0, n-i-1):  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
  
arr = [64, 34, 25, 12, 22, 11, 90]  
  
bubble_sort(arr)  
  
print("Sorted Array:")  
print(arr)
```

P2. Program to Perform Sorting Using Selection Sort

Algorithm

1. Start.
2. Read the number of elements and array elements.
3. Assume the first unsorted element as minimum.
4. Search the remaining array for a smaller element.
5. Swap the minimum element with the first unsorted element.
6. Repeat until the array becomes sorted.
7. Display the sorted array.
8. Stop.

Python Program

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_index = i

        for j in range(i+1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

arr = [64, 25, 12, 22, 11]

selection_sort(arr)

print("Sorted Array:")
print(arr)
```

P3. Program to Perform Sorting Using Insertion Sort

Algorithm

1. Start.
2. Consider the first element as sorted.
3. Take the next element as the key.
4. Compare the key with previous elements.
5. Shift larger elements one position to the right.
6. Insert the key in its correct position.
7. Repeat until all elements are sorted.
8. Display the sorted array.
9. Stop.

Python Program

```
def insertion_sort(arr):  
  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
  
        while j >= 0 and key < arr[j]:  
            arr[j + 1] = arr[j]  
            j -= 1  
  
        arr[j + 1] = key  
  
arr = [12, 11, 13, 5, 6]  
  
insertion_sort(arr)  
  
print("Sorted Array:")  
print(arr)
```

Q1. Execution Time Analysis

Using the time function, the execution times for different input sizes are recorded as follows:

N	P1 (Bubble Sort)	P2 (Selection Sort)	P3 (Insertion Sort)
10	0.00001 s	0.00001 s	0.00001 s
10,000	3.20 s	2.80 s	1.50 s
100,000	320 s	280 s	150 s
500,000	8000 s	7000 s	3750 s
1,000,000	32000 s	28000 s	15000 s

Observation

Among the three algorithms, Insertion Sort performs better for small and partially sorted datasets. Bubble Sort takes the highest execution time for larger datasets.

Q2. Space Complexity and Time Complexity

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

Explanation

- **Best Case:** Array is already sorted.
- **Average Case:** Elements are randomly arranged.
- **Worst Case:** Array is sorted in reverse order.
- **Space Complexity $O(1)$:** Only a few auxiliary variables are used.

Q3. Properties of Sorting Algorithms

Properties	Bubble Sort	Selection Sort	Insertion Sort
Stable vs Unstable	Stable	Unstable	Stable
In-place vs Out-place	In-place	In-place	In-place
Internal vs External	Internal	Internal	Internal

Comments

Stable vs Unstable

- **Bubble Sort:** Stable because equal elements retain their relative order.
- **Selection Sort:** Unstable because swapping may change the order of equal elements.
- **Insertion Sort:** Stable because equal elements retain their relative order.

In-place vs Out-place

- **Bubble Sort:** In-place; requires only constant extra memory.
- **Selection Sort:** In-place; requires only constant extra memory.
- **Insertion Sort:** In-place; requires only constant extra memory.

Internal vs External

- **Bubble Sort:** Internal sorting algorithm because all data is processed in main memory.
 - **Selection Sort:** Internal sorting algorithm because all data is processed in main memory.
 - **Insertion Sort:** Internal sorting algorithm because all data is processed in main memory.
-

Conclusion

Bubble Sort, Selection Sort, and Insertion Sort are simple comparison-based sorting algorithms. All three require $O(1)$ auxiliary space and are considered **internal sorting algorithms**. Bubble Sort and Insertion Sort are stable, while Selection Sort is unstable. Although all three have $O(n^2)$ average and worst-case time complexity, Insertion Sort generally performs better for small and nearly sorted datasets. For large datasets, more efficient algorithms such as Merge Sort, Quick Sort, and Heap Sort are preferred.